☁️ AWS Core Services — Foundations Study Guide

> **Focus Area:** AWS Compute, Storage & Networking Fundamentals **Objective:** Understand and hands-on configure EC2, Security Groups, AMIs, EBS, Snapshots, and Network Interfaces from scratch.

📋 Table of Contents

🖥️ 1. EC2 — Elastic Compute Cloud

💡 What is EC2?

Amazon EC2 (Elastic Compute Cloud) is a service that lets you rent virtual servers in the cloud on demand. Instead of buying physical hardware, you spin up a virtual machine in minutes, configure it exactly how you want, and pay only for the time you actually use it.

A practical way to picture it — imagine you need a powerful computer for a project. Instead of spending thousands buying one, you rent one from AWS, use it for a few hours or months, and stop paying the moment you're done.

EC2 handles the physical hardware, data center, and power supply. You handle the operating system, software, and what runs on it.

```
Traditional Setup vs EC2:

Physical Server:                EC2 Instance:
─────────────────               ──────────────
Buy hardware upfront     →      Launch in 60 seconds
Wait weeks to deploy     →      Pay per hour/second
Fixed capacity           →      Scale up or down anytime
```

```
You maintain hardware    →    AWS maintains hardware
High upfront cost        →    Zero upfront cost
```

🔩 Core Components

Instance Families

AWS groups instance types into families based on what they're optimized for:

Family	Optimized For	Common Types	Example Workload
General Purpose	Balanced CPU + RAM	t2.micro, t3.small, m5.large	Web servers, dev environments
Compute Optimized	High CPU performance	c5.xlarge, c6g.large	Gaming servers, HPC, video encoding
Memory Optimized	Large RAM capacity	r5.large, x1e.xlarge	In-memory databases, real-time analytics
Storage Optimized	High disk I/O	i3.large, d3.xlarge	Data warehouses, Hadoop clusters
Accelerated	GPU / FPGA	p3.xlarge, g4dn.large	Machine learning, 3D rendering

> **t2.micro** is free tier eligible — perfect for learning and small projects.

Instance Lifecycle States

```
Launch Request
     │
     ▼
  PENDING ——— (booting up, not charged)
     │
     ▼
  RUNNING ——— (fully operational, charged per second)
     │
     ├——— Stop ———▶ STOPPING ——▶ STOPPED (EBS kept, charged for storage only)
     │                                  │
     │                                  └——— Start ——▶ RUNNING
     │
     └——— Terminate ——▶ TERMINATED (permanent, data gone if no EBS)
```

State	What's Happening	Are You Charged?
🟡 Pending	Instance is starting up	No
🟢 Running	Fully operational	Yes — per second
🔵 Stopping	Shutting down in progress	No
⚫ Stopped	Off — EBS volumes retained	EBS storage cost only
🔴 Terminated	Permanently deleted	No

🚀 Hands-On: Launch Your First EC2 Instance

What We're Building

```
Your Laptop
     │
     │  SSH (port 22)
     ▼
EC2 Instance (Amazon Linux)
  Public IP: 13.x.x.x
  Type: t2.micro
  Storage: 8GB gp3
  Region: ap-south-1
```

Step-by-Step

1. Open EC2 Dashboard

- AWS Console → Search "EC2" → Click EC2 Dashboard
- Click the orange **"Launch Instance"** button

2. Configure Instance Details

Field	Value
Name	MyFirstWebServer
AMI	Amazon Linux 2023 (Free Tier Eligible)
Instance Type	t2.micro (Free Tier Eligible)
Key Pair	Create new → Name: `my-keypair` → Download `.pem` file

> ⚠️ The `.pem` file is your only way to SSH in. Save it somewhere safe — AWS will never show it again.

3. Network & Security Settings

Field	Value
VPC	Default VPC
Subnet	Any available subnet
Auto-assign Public IP	Enable
Security Group	Create new → Allow SSH (22), HTTP (80), HTTPS (443)

4. Storage

Field	Value
Root Volume	8 GB gp3 (default)
Delete on termination	Yes (default)

5. Launch and Connect

```
# Fix key permissions (required on Linux/Mac)
chmod 400 my-keypair.pem

# SSH into instance
ssh -i my-keypair.pem ec2-user@YOUR_PUBLIC_IP

# Verify you're inside the instance
whoami
# Output: ec2-user
```

🔒 2. Security Groups

💡 What is a Security Group?

A **Security Group** is a set of firewall rules that controls what network traffic is allowed to reach your EC2 instance and what traffic can leave it. Every EC2 instance must have at least one Security Group — there's no way around it.

Think of a Security Group like a bouncer at a nightclub. The bouncer has a list of who gets in (inbound rules) and checks people as they arrive. If you're not on the list, you're not getting in — regardless of how you got to the door.

```
Inbound Traffic (from internet/other instances)
        |
```

```
        ▼
┌──────────────────────────┐
│     Security Group       │
│                          │
│  Rule 1: Allow port 22   │ ← SSH from your IP only
│  Rule 2: Allow port 80   │ ← HTTP from anywhere
│  Rule 3: Allow port 443  │ ← HTTPS from anywhere
│  Everything else: DENY   │ ← Implicit deny (not listed)
└──────────────────────────┘
        │
        ▼
   EC2 Instance
```

⚙️ How Security Groups Work

Core Rules:

```
Default Behavior:
─────────────────
Inbound  → ALL DENIED by default (you open only what you need)
Outbound → ALL ALLOWED by default (instances can reach internet)

Key Properties:
───────────────
✅ Stateful        – allow inbound port 80, response automatically allowed
✅ Allow-only      – no DENY rules exist, only ALLOW rules
✅ Multi-attach    – one instance can have multiple security groups
✅ Instant effect  – rule changes apply without restarting instance
✅ SG referencing  – use another SG as source instead of an IP address
```

Stateful Explained:

```
Without stateful (how NACLs work):
  You open port 80 inbound ✅
  You must also open ephemeral ports outbound for response ⚠️

With stateful (how Security Groups work):
  You open port 80 inbound ✅
  Response traffic automatically allowed ✅ (no extra rule needed)
```

📋 Rule Fields Explained

Field	What It Means	Example Value
Type	Predefined protocol shortcut	SSH, HTTP, MySQL/Aurora

Field	What It Means	Example Value
Protocol	Network protocol	TCP, UDP, ICMP
Port Range	Which port(s) to open	22, 80, 443, 3306, 0-65535
Source (Inbound)	Who is allowed to connect	`0.0.0.0/0`, `My IP`, another SG
Destination (Outbound)	Where traffic can go	`0.0.0.0/0`

Source Options Explained:

```
0.0.0.0/0        → Anyone on the internet (use carefully)
My IP            → Only your current IP address
10.0.0.0/16      → Any IP within your VPC
sg-xxxxxxxx      → Only instances that have this specific Security Group
```

🚀 Hands-On: Create and Attach a Security Group

Step-by-Step

1. Create the Security Group

- EC2 Dashboard → Left sidebar → **Security Groups**
- Click "**Create security group**"

Field	Value
Name	WebServer-SG
Description	Allow HTTP, HTTPS from everywhere, SSH from my IP only
VPC	Select your VPC

2. Add Inbound Rules

Type	Protocol	Port	Source	Purpose
SSH	TCP	22	My IP	Admin access — your IP only
HTTP	TCP	80	0.0.0.0/0	Web traffic from anyone
HTTPS	TCP	443	0.0.0.0/0	Secure web traffic from anyone

3. Outbound Rules

- Leave default: All traffic → 0.0.0.0/0

- This lets your instance download updates, call APIs, etc.

4. Attach to an Existing Instance

```
EC2 → Instances → Select your instance
→ Actions → Security → Change security groups
→ Search and select "WebServer-SG"
→ Click "Save"
```

> Changes apply instantly — no reboot needed.

🖼️ 3. AMI — Amazon Machine Image

💡 What is an AMI?

An **Amazon Machine Image (AMI)** is a saved template of an EC2 instance — essentially a complete snapshot of everything needed to recreate that exact server from scratch. When you launch an EC2 instance, you're always launching from an AMI.

Real-world analogy: Imagine you spent a week setting up a perfect workstation — installed all your tools, configured everything exactly right, personalized every setting. An AMI is like cloning that entire workstation so you (or your team) can spin up identical copies instantly, anytime, without repeating all that setup work.

```
One AMI → Launch 1 instance   (single server)
One AMI → Launch 10 instances (instant fleet)
One AMI → Launch in any region (global deployment)
```

📦 What an AMI Contains

```
AMI Contents:
──────────────────────────────────────────
📀 Root Volume Snapshot
   └── Operating system (Linux, Windows)
   └── All installed software (NGINX, Node.js, MySQL)
   └── All your custom configurations

🔐 Launch Permissions
   └── Which AWS accounts can use this AMI

💾 Block Device Mapping
   └── Which EBS volumes to attach on launch
   └── Volume sizes and types
```

```
⚙️ Metadata
   └── Architecture: x86_64 or ARM
   └── Virtualization type: HVM
   └── Region where AMI lives
```

🗂️ AMI Types

Category	Created By	Best For	Example
🔴 **AWS Provided**	Amazon	Starting fresh, official base OS	Amazon Linux 2023, Ubuntu 22.04
🛒 **AWS Marketplace**	Third-party vendors	Pre-packaged software stacks	Bitnami WordPress, Palo Alto Firewall
👥 **Community AMIs**	Public AWS users	Specialized or uncommon setups	Custom distros, research images
🔧 **Your Custom AMIs**	You	Pre-configured app environments	Your app server fully configured

🌎 AMI Regions and Sharing

```
AMI Scope Rules:
──────────────────────────────────────────────
❌ AMI in ap-south-1 CANNOT directly launch in us-east-1
✅ Use "Copy AMI" to replicate it to another region

Sharing Options:
──────────────────────────────────────────────
🔒 Private (default)      → Only your AWS account
👤 Specific accounts      → Share with exact AWS Account IDs
🌐 Public                 → Anyone in the world can use it
```

When to copy AMIs across regions:

- Setting up disaster recovery in a secondary region
- Deploying identical servers globally for lower latency
- Sharing a pre-built environment with a team in another region

🚀 Hands-On: Create a Custom AMI and Share It

What We're Doing

```
Configured EC2 Instance
        │
        │  Create Image
        ▼
Custom AMI (saved template)
        │
        ├──── Launch new identical instance anytime
        ├──── Share with another AWS account
        └──── Copy to another region
```

Step-by-Step

1. Prepare Your Instance

Make sure everything is installed and configured:

```
# Example: install and verify NGINX
sudo yum update -y
sudo yum install nginx -y
sudo systemctl start nginx
sudo systemctl enable nginx
nginx -v
```

2. Create the AMI

- EC2 → Instances → Select your configured instance
- **Actions → Image and Templates → Create Image**

Field	Value
Image Name	MyApp-AMI-v1
Description	NGINX web server — configured July 2025
No Reboot	Leave unchecked (checked = risky, may cause inconsistency)

- Click "**Create Image**"
- Navigate to **EC2 → AMIs** — status shows `Pending` → wait for `Available` (3–10 min)

3. Share the AMI

- Select your AMI → **Actions → Edit AMI Permissions**
- Change from Public to Private
- Under "**Add AWS Account ID**" → enter the target account's 12-digit ID
- Click "**Add permission**" → "**Save changes**"

4. Copy AMI to Another Region

- Select AMI → **Actions** → **Copy AMI**
- Select destination region (e.g., `us-east-1`)
- Click "**Copy AMI**"
- Switch to that region and check under AMIs — it will appear there

💾 4. EBS — Elastic Block Store

💡 What is EBS?

Amazon Elastic Block Store (EBS) provides persistent storage volumes that you attach to EC2 instances. Think of EBS volumes exactly like hard drives or SSDs — they store data, survive reboots, and can be detached and reattached to different instances.

The most important thing about EBS: **data persists independently from the instance lifecycle**. If you stop your EC2 instance and restart it tomorrow, all your EBS data is still there exactly as you left it. This is completely different from instance store (ephemeral storage), which vanishes the moment an instance stops.

```
EBS vs Instance Store:

EBS Volume:
  Instance stops → Data SAFE ✅
  Instance starts → Data still there ✅
  Instance terminates → Data SAFE (if "delete on termination" = No) ✅

Instance Store (ephemeral):
  Instance stops → Data GONE ❌
  Instance terminates → Data GONE ❌
  Good only for temporary scratch data
```

📊 EBS Volume Types

Type	Category	Max IOPS	Max Throughput	Best For
gp3	SSD	16,000	1,000 MB/s	Default choice — OS volumes, apps, dev
gp2	SSD	16,000	250 MB/s	Legacy general purpose (prefer gp3)

Type	Category	Max IOPS	Max Throughput	Best For
io2 Block Express	SSD	256,000	4,000 MB/s	Critical databases — Oracle, SAP HANA
io1	SSD	64,000	1,000 MB/s	High I/O apps needing guaranteed performance
st1	HDD	500 MB/s throughput	500 MB/s	Big data, log processing, Kafka
sc1	HDD	250 MB/s throughput	250 MB/s	Cold archives, rarely accessed backups

> **Rule of thumb:** Use `gp3` for almost everything. Switch to `io2` only when your database demands guaranteed ultra-high IOPS.

🔑 Important EBS Characteristics

```
🔒 AZ-Locked:
   EBS volume in ap-south-1a → can ONLY attach to EC2 in ap-south-1a
   To move to 1b → take snapshot → create new volume in 1b from snapshot

🔗 One-to-One Attachment:
   One EBS volume → One EC2 instance at a time
   Exception: io1/io2 support Multi-Attach (advanced use case)

📏 Elastic Sizing:
   Increase size → No downtime needed
   Change type → No downtime needed (e.g., gp2 → gp3)
   Decrease size → NOT supported (can only grow, not shrink)

🔐 Encryption:
   Uses AES-256 encryption
   Encrypting a volume also encrypts all snapshots made from it
   Encrypted volumes → encrypted AMIs → encrypted new volumes
```

🚀 Hands-On: Create, Attach, and Mount an EBS Volume

What We're Building

```
EC2 Instance
├── /dev/xvda  → Root volume (OS) — 8GB gp3
```

```
└── /dev/xvdf  → New data volume — 20GB gp3  ← We're adding this
                 Mounted at: /data
```

Step-by-Step

1. Create the EBS Volume

- EC2 Dashboard → Elastic Block Store → **Volumes** → **Create volume**

Setting	Value
Volume Type	gp3
Size	20 GiB
Availability Zone	⚠️ MUST match your EC2 instance's AZ (e.g., ap-south-1a)
IOPS	3000 (default gp3)
Throughput	125 MB/s (default)
Tag Name	DataVolume-01

- Click "**Create volume**" → Status shows `Available`

2. Attach the Volume to Your Instance

- Select the volume → **Actions** → **Attach volume**
- Select your EC2 instance from dropdown
- Device name: `/dev/xvdf`
- Click "**Attach volume**" → Status changes to `In-use`

3. Format and Mount Inside EC2

```
# SSH into your instance first
ssh -i my-keypair.pem ec2-user@YOUR_PUBLIC_IP

# Verify the new disk is visible
sudo lsblk
# You should see xvdf listed

# Format the volume (only do this on a NEW, empty volume)
sudo mkfs -t ext4 /dev/xvdf

# Create a directory to mount it
sudo mkdir /data

# Mount the volume
sudo mount /dev/xvdf /data
```

```
# Verify it's mounted
df -h | grep /data
# Should show 20G available

# Make it auto-mount after every reboot
echo '/dev/xvdf /data ext4 defaults,nofail 0 2' | sudo tee -a /etc/fstab

# Test auto-mount config
sudo mount -a
```

> ⚠️ Only run `mkfs` on a brand new empty volume. Running it on a volume with data will erase everything.

📸 5. EBS Snapshots

💡 What is an EBS Snapshot?

An **EBS Snapshot** is a point-in-time backup of an EBS volume stored durably in Amazon S3 (managed behind the scenes by AWS — you won't see it as an S3 bucket). It captures the exact byte-for-byte state of your volume at the moment you triggered the snapshot.

Practical example: Your EC2 instance runs a production database. Every night at midnight, you take a snapshot. If something goes catastrophically wrong on Tuesday afternoon, you can restore from Monday night's snapshot and only lose one day's data — rather than everything.

```
EBS Volume Timeline:

Monday midnight  → Snapshot 1 taken ✅
Tuesday 2pm      → Ransomware attack 😱
Tuesday 2:01pm   → Restore from Snapshot 1 ✅
Result: Only lost ~14 hours of data, not everything
```

🔄 How Incremental Snapshots Work

After the first full snapshot, AWS only saves what changed — not the entire volume each time.

```
Volume Size: 100 GB

Snapshot 1 (First ever):
  └── Copies ALL 100GB → Stored: 100 GB

Snapshot 2 (Next day, 15GB changed):
  └── Copies only 15GB changed blocks → Stored: 15 GB
```

```
Snapshot 3 (Day after, 8GB changed):
  └── Copies only 8GB changed blocks → Stored: 8 GB

Total storage used: 100 + 15 + 8 = 123 GB
Without incremental: 100 + 100 + 100 = 300 GB
Savings: 177 GB saved! 💰
```

> Deleting an older snapshot does NOT break newer ones. AWS reorganizes the data blocks automatically.

🛠️ What You Can Do with Snapshots

```
EBS Snapshot
     │
     ├── 🔄 Restore → Create new EBS volume from snapshot
     │                (recover data, move to different AZ)
     │
     ├── 🌍 Copy → Send to another AWS region
     │             (disaster recovery, geographic redundancy)
     │
     ├── 🤝 Share → Grant access to specific AWS accounts
     │              (hand off data to partners/teams)
     │
     ├── 🖼️ AMI → Create AMI from a root volume snapshot
     │            (package an OS + software image)
     │
     └── ⏰ Automate → Use Data Lifecycle Manager (DLM)
                       (scheduled automatic snapshots, no manual work)
```

🚀 Hands-On: Create, Restore, and Copy Snapshots

Step-by-Step

1. Create a Snapshot

- EC2 → Elastic Block Store → **Volumes**
- Select the volume you want to back up
- **Actions → Create snapshot**

Field	Value
Description	Pre-upgrade backup — July 2025
Tag Name	Pre-Upgrade-Snap-01

- Click "**Create snapshot**"
- Navigate to **EC2** → **Snapshots** → watch status: `Pending` → `Completed`

2. Restore from Snapshot (Create New Volume)

- Select the snapshot → **Actions** → **Create volume from snapshot**

Setting	Value
Volume Type	gp3
Size	Same or larger than original
Availability Zone	Target AZ for the new volume

- Click "**Create volume**" → attach to an instance as needed

3. Copy Snapshot to Another Region

- Select snapshot → **Actions** → **Copy snapshot**
- Select destination region (e.g., `us-east-1`)
- Click "**Copy snapshot**" → switch to that region to monitor

4. Share Snapshot with Another Account

- Select snapshot → **Actions** → **Modify permissions**
- Add the target AWS Account ID
- Click "**Save changes**"
- The other account finds it under **Private snapshots** in their console

🌐 6. Network Interfaces & IP Addressing

💡 What is an ENI?

An **Elastic Network Interface (ENI)** is a virtual network card that attaches to an EC2 instance. Just like a physical server needs a network card to connect to a network, an EC2 instance needs an ENI to connect to a VPC.

Every EC2 instance gets one ENI automatically on launch — called the **primary network interface**. You can attach additional ENIs for advanced setups like dual-homed instances (connected to two subnets simultaneously) or failover configurations.

```
EC2 Instance
├── Primary ENI (eth0)     ← Always present, created automatically
│   ├── Private IP: 10.0.1.25
```

```
│   ├── Public IP: 13.x.x.x  (if in public subnet)
│   └── Security Groups: [WebServer-SG]
│
└── Secondary ENI (eth1)   ← Optional, you attach manually
    ├── Private IP: 10.0.2.30
    └── Security Groups: [App-SG]
```

ENI Failover Use Case:

```
Server A fails:
  ENI (with static private IP) detaches from Server A
       ↓
  ENI reattaches to Server B
       ↓
  Server B now has the same private IP as Server A had
       ↓
  All internal traffic continues without IP changes ✅
```

🌎 The Three Types of IP Addresses

```
Your EC2 Instance has up to 3 types of IPs:

┌──────────────────────────────────────────────────────┐
│                   EC2 Instance                       │
│                                                      │
│   🔵 Private IP: 10.0.1.25                           │
│      Always present, internal VPC only               │
│                                                      │
│   🟡 Public IP: 54.12.34.56                          │
│      Auto-assigned, changes on stop/start            │
│                                                      │
│   🟢 Elastic IP: 13.110.0.5                          │
│      Static, you own it, never changes               │
└──────────────────────────────────────────────────────┘
```

🔵 Private IP

Every EC2 instance in a VPC is assigned a private IP from the VPC's CIDR range automatically. This IP is used for all traffic that stays inside AWS — instance-to-instance communication, talking to RDS databases, connecting to load balancers, etc.

```
Private IP Properties:
──────────────────────

Assigned from VPC CIDR (e.g., 10.0.0.0/16)
```

```
Stays the same when instance is stopped and restarted ✅
NOT reachable from the public internet ❌
Used for all internal VPC communication
AWS provides internal DNS: ip-10-0-1-25.ec2.internal
Can assign multiple private IPs to one ENI
```

🟡 Public IP

Automatically assigned when an EC2 instance launches in a public subnet (if auto-assign is enabled). Allows direct internet access through the Internet Gateway.

```
⚠️ Critical Behavior:

Instance Running  → Public IP: 54.12.34.56
Instance STOPPED  → Public IP: GONE ❌
Instance STARTED  → Public IP: 54.98.11.200 (completely different!)

This means:
  → DNS records pointing to this IP break
  → Hardcoded IPs in configs break
  → Use Elastic IP if you need a stable address
```

🟢 Elastic IP (EIP)

An **Elastic IP** is a static public IPv4 address that you reserve from AWS and own until you explicitly release it. Unlike the auto-assigned public IP, an Elastic IP stays yours regardless of what happens to the underlying instance.

```
Elastic IP Benefits:
──────────────────────────────────────────────
✅ Never changes — stop, start, even replace instance
✅ Can be moved between instances instantly
✅ Great for production servers that need a fixed IP
✅ Works for DNS A records (IP stays the same forever)
✅ Useful for whitelisting (partners allow your fixed IP)

Elastic IP Cost Warning:
──────────────────────────────────────────────
💰 FREE when attached to a RUNNING instance
💸 $0.005/hour when NOT attached (or instance stopped)

Always release EIPs you're not using!
5 EIPs per region per account (default limit)
```

Elastic IP Failover Pattern:

```
Normal:          Server A ←— Elastic IP (3.110.x.x)

Server A fails:  Server A ❌

Recovery:        Server B ←— Elastic IP (3.110.x.x) ← same IP!

Result: Your DNS, firewall rules, partner whitelists — nothing breaks ✅
```

📊 IP Types Side-by-Side

Feature	🔵 Private IP	🟡 Public IP	🟢 Elastic IP
Who Assigns	AWS automatically	AWS automatically	You request it
Survives Stop/Start	✅ Yes	❌ No — changes	✅ Yes
Internet Accessible	❌ No	✅ Yes	✅ Yes
Cost	Free	Free while running	Free if attached and running
Use Case	Internal VPC traffic	Temporary dev/test access	Production servers, DNS
Can Be Static	N/A (already stable)	❌ No	✅ Yes

🚀 Hands-On: Allocate and Use an Elastic IP

Step-by-Step

1. Allocate an Elastic IP

- EC2 Dashboard → Network & Security → **Elastic IPs**
- Click "**Allocate Elastic IP address**"
- Network border group: Leave default (your current region)
- Click "**Allocate**"
- AWS assigns you a static IP like `3.110.x.x` — this is now yours

2. Associate the EIP with Your Instance

- Select the newly allocated EIP
- **Actions → Associate Elastic IP address**

Field	Value
Resource type	Instance
Instance	Select your EC2 instance
Private IP	Select the primary private IP

- Click "**Associate**"
- Go to EC2 → Instances → your instance now shows the EIP as its public IPv4 ✅

3. Verify the EIP Sticks After Restart

```
# Note the current EIP
# Example: 3.110.45.67

# Stop the instance
# Wait 30 seconds
# Start the instance

# Check the public IP again in console
# It should still show: 3.110.45.67 ✅
# (contrast this with a regular public IP which would change)
```

4. Release an Elastic IP When Done

```
⚠️ Important — releasing unused EIPs saves money

EC2 → Elastic IPs → Select EIP
→ Actions → Disassociate   (detach from instance first)
→ Actions → Release Elastic IP address
→ Confirm

The IP returns to AWS pool. Billing stops immediately.
```

⚖️ Auto Scaling Groups

💡 What is an Auto Scaling Group?

An **Auto Scaling Group (ASG)** is an AWS service that automatically manages the number of EC2 instances running your application. It watches your infrastructure, responds to changes in demand, and keeps your application available — all without any manual intervention.

Think of it like a smart staffing agency for your servers. When your website gets a traffic spike, the agency automatically hires more servers. When traffic drops at night, it lets some go. You define the rules, it handles the execution.

```
Traffic Pattern vs Instance Count:

9 AM  — Office opens, traffic spikes
        ASG detects CPU > 70%
        Launches 2 new instances ↑

2 PM  — Steady traffic
        ASG maintains current count
        No change needed

11 PM — Traffic drops overnight
        ASG detects CPU < 30%
        Terminates extra instances ↓

Result: You always have exactly what you need ✅
        No over-provisioning, no under-provisioning
```

🔑 Core Concepts

Term	What It Means
Launch Template	The blueprint ASG uses to create each new instance (AMI, type, SG, startup script)
Desired Capacity	The number of instances ASG actively tries to maintain right now
Minimum Capacity	Hard floor — ASG will never go below this count, even at zero load
Maximum Capacity	Hard ceiling — ASG will never exceed this, even at peak load
Scaling Policy	The rules that decide when to add or remove instances
Health Check	ASG continuously checks instances — unhealthy ones get replaced automatically

```
Capacity Boundaries Visualized:

Maximum: 8  ████████ ← ASG never exceeds this
                ↕  scaling happens here
Desired:  4  ████     ← Current target
```

```
                    ↕  scaling happens here
Minimum:  2  ▓▓          ← ASG never drops below this
```

📈 Scaling Policy Types

Policy	How It Decides to Scale	Best For
Target Tracking	Keeps a specific metric at your target (e.g., CPU stays at 50%)	Most common — simple and effective
Step Scaling	Scales by defined amounts at different alarm thresholds	Fine-grained control over scale steps
Scheduled	Scales at specific times you define (e.g., 9 AM add 3, 10 PM remove 3)	Predictable traffic patterns
Predictive	Uses ML to forecast traffic and pre-scales ahead of time	Large applications with historical data

```
Target Tracking Example:

Target: CPU = 50%

CPU hits 75% → Too high → ASG adds instances → CPU drops back toward 50%
CPU drops 20% → Too low → ASG removes instances → CPU rises back toward 50%
CPU stays 50% → Perfect → ASG does nothing
```

🏗️ Why Use an ASG?

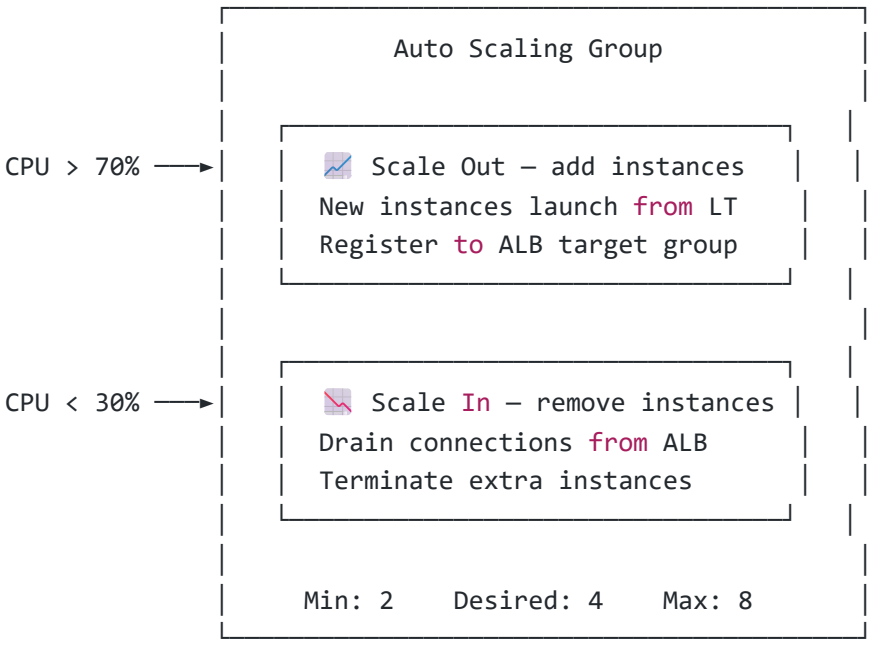
```
Without ASG:                    With ASG:
─────────────                   ──────────
Traffic spike → site crashes    Traffic spike → new instances launch ✅
Low traffic → paying for idle   Low traffic → idle instances removed ✅
Instance dies → site is down    Instance dies → replaced in minutes ✅
You manually scale at 3am       ASG scales while you sleep ✅
Fixed monthly cost              Pay only for what you use ✅
```

Four Core Benefits:

- 🟢 **High Availability** — Failed instances are detected and replaced automatically without any downtime
- 💰 **Cost Efficiency** — You scale in during quiet periods and scale out only when demand requires it

- 🤖 **Zero Manual Work** — Load-based decisions happen automatically based on your defined rules
- 🔀 **ALB Integration** — New instances register themselves to target groups and start receiving traffic immediately

⚙️ ASG Scaling Flow Diagram



🚀 Hands-On: Create a Launch Template

A Launch Template is the instruction set ASG follows every time it needs to create a new instance. Get this right and every auto-launched instance is identical and production-ready from second one.

What the Launch Template Defines

```
Launch Template: web-server-lt
──────────────────────────────────────────
AMI           → Which OS + software image
Instance Type → How powerful each server is
Key Pair      → SSH access credentials
Security Group → Firewall rules
Storage       → Disk size and type
User Data     → Startup script that runs on first boot
```

Step-by-Step

1. Open Launch Templates

- EC2 Dashboard → Left sidebar → **Launch Templates**
- Click **"Create launch template"**

2. Template Details

Field	Value
Launch template name	web-server-lt
Version description	v1 — Apache web server
Auto Scaling guidance	✅ Check this box

3. AMI and Instance

Field	Value
AMI	Amazon Linux 2023 (Free Tier Eligible)
Instance type	t3.small
Key pair	Select your existing key pair

4. Network Settings

Field	Value
Subnet	Do NOT specify — ASG will choose
Security groups	Select `web-sg`

> Leaving subnet blank lets the ASG distribute instances across multiple AZs — critical for high availability.

5. Storage

Field	Value
Volume type	gp3
Size	8 GiB

6. User Data Script (Advanced Details → User Data)

```
#!/bin/bash

# Update system packages
apt update -y
```

```
# Install Apache web server
apt install -y apache2

# Start and enable Apache
systemctl start apache2
systemctl enable apache2

# Fetch instance ID from metadata service (IMDSv2)
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
  -H "X-aws-ec2-metadata-token-ttl-seconds: 21600")

INSTANCE_ID=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
  http://169.254.169.254/latest/meta-data/instance-id)

# Create a page that shows which instance is serving the request
echo "<h1>Response from Instance: $INSTANCE_ID</h1>" > /var/www/html/index.html
```

> **What this script does:** Every time ASG launches a new instance, this script runs automatically. It installs Apache, starts it, and creates a webpage showing the instance ID — so you can visually confirm load balancing is working.

- Click "**Create launch template**" ✅

🚀 Hands-On: Create the Auto Scaling Group

Step-by-Step

Step 1 — Name and Launch Template

- EC2 → **Auto Scaling Groups** → **Create Auto Scaling Group**

Field	Value
Auto Scaling group name	web-asg
Launch template	web-server-lt
Version	Latest (always uses newest template version)

Step 2 — Network Configuration

Field	Value
VPC	Your production VPC
Availability Zones / Subnets	`app-private-subnet-1a` AND `app-private-subnet-1b`

> Always select **at least 2 subnets in different AZs**. If one AZ goes down, your instances in the other AZ keep serving traffic.

```
ASG Multi-AZ Layout:

AZ: ap-south-1a          AZ: ap-south-1b
┌──────────────┐         ┌──────────────┐
│  Instance 1  │         │  Instance 2  │
│  Instance 3  │         │  Instance 4  │
└──────────────┘         └──────────────┘
        └──────── ALB routes to both ────────┘

If 1a goes down → 1b still serves all traffic ✅
```

Step 3 — Load Balancer Integration

Field	Value
Load balancing	Attach to an existing load balancer
Target group	Select `web-tg`
Health checks	✅ Enable Elastic Load Balancing health checks
Health check grace period	300 seconds

> **Why 300 seconds grace period?** When a new instance launches, it needs time to finish the user data script, start Apache, and become ready. Without this grace period, ASG might mark it unhealthy before it's done booting and terminate it immediately.

Step 4 — Capacity and Scaling

Setting	Value
Desired capacity	2
Minimum capacity	2
Maximum capacity	6
Scaling policy type	Target tracking
Metric	Average CPU Utilization
Target value	50%

```
What Target Tracking at 50% CPU means:

2 instances running, CPU hits 75% average:
  → CloudWatch alarm triggers
  → ASG launches 1-2 more instances
  → Load spreads across more instances
  → CPU average drops back toward 50%

Traffic decreases, CPU drops to 20%:
  → ASG identifies excess instances
  → Drains connections via ALB
  → Terminates extra instances
  → CPU rises back toward 50%
```

Step 5 — Notifications (Optional but Recommended)

- Add an SNS topic to receive email alerts when ASG launches or terminates instances
- Useful for auditing and keeping track of scaling events in production

Step 6 — Review and Create

- Review all settings
- Click "**Create Auto Scaling Group**" ✅

✅ Verify the ASG is Working

Check Scaling Activity:

```
EC2 → Auto Scaling Groups → Select web-asg → Activity tab

You should see entries like:
  ✅ Launching instance i-0abc123 — Successful
  ✅ Launching instance i-0def456 — Successful
```

Check Instances Were Created:

```
EC2 → Instances

Filter by ASG name tag → web-asg
You should see 2 instances in Running state
Both should be in different AZs (1a and 1b)
```

Check Target Group Registration:

```
EC2 → Target Groups → web-tg → Targets tab

Both instances should appear as:
  Status: Healthy ✅
  Port: 80
```

🧪 Test Auto Scaling — Simulate a CPU Spike

This test confirms your scaling policy actually works by artificially driving up CPU and watching ASG respond.

SSH into one of the ASG instances:

```
ssh -i my-keypair.pem ec2-user@INSTANCE_PUBLIC_IP
```

Install the stress testing tool:

```
sudo yum install -y stress
```

Run CPU stress test:

```
# Stress 4 CPU cores for 5 minutes
stress --cpu 4 --timeout 300
```

Watch what happens in the console:

```
1. CloudWatch picks up rising CPU metric
        ↓
2. CPU average crosses 50% threshold
        ↓
3. CloudWatch alarm state changes to ALARM
        ↓
4. ASG receives scale-out signal
        ↓
5. New instance launches from web-server-lt
        ↓
6. New instance registers to web-tg
        ↓
7. ALB starts routing traffic to it
        ↓
8. CPU load spreads → metric drops back toward 50%
```

Monitor in real time:

```
EC2 → Auto Scaling Groups → web-asg
→ Activity tab            ← Watch new launch events appear
→ Monitoring tab          ← Watch CPU metric graph
→ Instance management     ← Watch new instances appear
```

After stress test ends:

```
CPU drops → alarm clears → scale-in eventually occurs
(scale-in has a cooldown period — usually 5-15 min after CPU drops)
```

📋 ASG Quick Reference

```
🔑 Key Numbers to Set:
   Min  = the floor you never drop below (for HA, set ≥ 2)
   Max  = the ceiling you never exceed (controls cost)
   Desired = where you start right now

⏱️  Timing to Know:
   Health check grace period: 300 sec (let instance finish booting)
   Scale-out cooldown: ~60-300 sec (wait before adding more)
   Scale-in cooldown: ~300 sec (wait before removing — prevents flapping)

✅  Always Do:
   → Spread across 2+ AZs (never single AZ for production)
   → Attach to a Target Group (ALB handles traffic distribution)
   → Enable ELB health checks (not just EC2 health checks)
   → Set meaningful Min ≥ 2 (single instance = single point of failure)

❌  Avoid:
   → Min = 0 (application goes completely down during low traffic)
   → Single AZ deployment (one AZ outage = full outage)
   → No scaling policy (defeats the purpose of ASG)
   → Grace period too short (instances get terminated before they're ready)
```

📚 Quick Reference Summary

🗂️ Service Overview

Service	One-Line Purpose
EC2	Rent virtual servers in the cloud, pay per second
Security Groups	Virtual firewall — control who can reach your instance
AMI	Saved server template — launch identical instances anytime
EBS	Persistent virtual hard drive attached to EC2
Snapshots	Point-in-time EBS backup stored in S3
ENI	Virtual network card that connects instance to VPC
Elastic IP	Your own static public IP that never changes
Autoscaling Groups	Increases resources based on requirements automatically

💡 Key Rules to Remember

```
🖥️ EC2
   └── Always launched from an AMI
   └── t2.micro is free tier — use it for learning

🔒 Security Groups
   └── Stateful — allow inbound, response is auto-allowed
   └── Allow-only — no DENY rules
   └── Changes apply instantly, no restart needed

🖼️ AMI
   └── Region-specific — copy to deploy in other regions
   └── Create BEFORE making major changes (safety net)

💾 EBS
   └── Locked to one AZ — must match EC2's AZ
   └── Data persists after stop/start (not termination by default)
   └── gp3 = best default choice for most workloads
   └── Can only grow in size, never shrink

📸 Snapshots
   └── Incremental after first full backup
   └── Stored in S3 (managed by AWS, not visible to you)
   └── Can copy across regions for disaster recovery

🌐 IP Addresses
   └── Private IP — stable, internal only, always present
   └── Public IP — free but changes on every stop/start
   └── Elastic IP — static, yours to keep, release when unused
   └── Unused EIP = money wasted ($0.005/hr)
```